

Flubble Linear Decomposition Proof Boundary

Rendered proof note for render-flubble-linear, 2026-06-12

Contents

1. Purpose	1
2. Mathematical Model	1
2.1. Hierarchy Semantics	2
2.2. Cost Model	2
3. Formally Proved In Lean	3
3.1. Flubble Definition And Input Contracts	3
3.2. Detector Correctness And Bounds	3
3.3. Indexed Detector And Equivalence	3
3.4. Hierarchy Correctness	4
3.5. Cost And Linearity Theorems	4
3.6. Semantic Pipeline Result	5
4. Conditional Linear-Time Theorem	5
5. C++ Bridge And Conformance Evidence	6
5.1. Cost Instrumentation	6
5.2. Flubble Export And Structure Evidence	6
5.3. Checked Translator	6
5.4. Cycle Class Checker	6
5.5. Hierarchy Span Checker	7
6. Remaining Open Gap	7
7. Build And Reference Check	7

1. Purpose

This note is a distributable mathematical description of the current flubble decomposition proof story in povu. It deliberately separates four layers:

1. facts formally proved in Lean,
2. conditional linear-time theorems and their stage contracts,
3. current C++/Rust bridge and conformance evidence, and
4. the remaining open gap to a full arbitrary C++ runtime refinement proof.

The formal boundary is semantic. Lean proves properties of proof-side graph, walk, detector, hierarchy, cost, and VCF emission models. Current C++ and Rust behavior is supported by conformance tests and bridge artifacts, but there is not yet a Lean theorem stating that every current povu execution refines the semantic model or satisfies the proof-side runtime-cost records.

2. Mathematical Model

Let a graph be represented by a Lean Graph with segment list V and directed link list E . The proof-side graph-size measure used by the cost layer is

$$n(G) = |V| + \text{edgeCount}(G).$$

A traversal frame $F : \text{TraversalFrame } G$ carries an ordered list of tree links. The flubble detector does not scan all links directly. It scans the candidate stack

$$S(F) = \text{candidateStack}(F) = [e_0, e_1, \dots, e_{m-1}],$$

the ordered subsequence of real black tree edges eligible to be flubble endpoints. A cycle-class assignment C classifies candidate links. Its formal correctness contract says class equality is sound and complete for the Lean cycle-equivalence relation on boundary candidates.

For candidate e_i , the base detector looks for the first later same-class candidate after a nonempty gap:

$$j = \min\{k \mid i + 1 < k < m \wedge C(e_i) = C(e_k)\}.$$

If such j exists, the candidate pair is canonicalized by edge id:

$$b = \text{Boundary.ordered}(e_i, e_j).$$

The decomposition relation for base flubbles is therefore:

$\text{IsFlubbleBoundary}(F, C, b) \Leftrightarrow b$ is a paper boundary and b arises from the canonical close-after-gap scan over $S(F)$.

The emitted boundary list is deduplicated and deterministic. The indexed detector introduces a class-index state so the same relation can be computed by one pass over $S(F)$: one lookup and one update per candidate.

2.1. Hierarchy Semantics

Each boundary has a span in the candidate stack. If

$$b = (e_i, e_j) \text{ with } i < j,$$

then $\text{boundarySpan? } S \ b = \text{some } \{ \text{start} := i, \text{finish} := j \}$. A hierarchy input is supported when every emitted boundary has a span and all spans are laminar: for every two distinct spans, one strictly contains the other or they are disjoint. Parent semantics is nearest strict span containment:

$$\text{parent}(b) = \arg \min_p \text{width}(p)$$

among spans p that strictly contain the span for b ; if no such span exists, b is a root of the flubble forest. Lean represents the hierarchy as a flat list of nodes with optional parent boundaries and proves it is a forest that preserves detector order, duplicate freedom, and canonical endpoint orientation.

2.2. Cost Model

The flubble extraction cost model is proof-side, not an observed timing model. It has four slots:

1. indexedOperations,
2. scanBookkeeping,
3. boundaryEmission, and
4. hierarchyConstruction.

The indexed operation contract supplies constant lookup/update costs and the exact count theorem that the indexed scan performs one lookup and one update per candidate. Boundary output size is bounded by $m = |S(F)|$. Under supported hierarchy input, flat hierarchy output size is bounded by $3m$: one node slot plus two reference metadata slots per detected boundary.

Thus Lean proves the conditional statement:

$$T_{\text{extract}(F, C)} \in O(|S(F)|),$$

assuming the explicit operation, bookkeeping, boundary-emission, and hierarchy construction cost contracts.

For the full semantic decomposition pipeline, Lean composes named stage costs:

1. component decomposition,
2. tip/dummy augmentation,
3. traversal-frame construction,
4. cycle-class assignment,
5. hairpin scan, and
6. flubble extraction plus hierarchy construction.

Given intermediate-size bounds back to $n(G)$, the composed theorem concludes:

$$T_{\text{decompose}(G)} \in O(n(G)).$$

This theorem is conditional on the named stage contracts. It is not a statement that current C++ povu has already been formally proved linear.

3. Formally Proved In Lean

The following names are Lean definitions or theorems in the Povulean library. They are the formal proof boundary this document relies on.

3.1. Flubble Definition And Input Contracts

Module: `PovuLean.Algorithms.Flubble.Spec`

Key names:

1. `PovuLean.Algorithms.Flubble.Boundary`
2. `PovuLean.Algorithms.Flubble.Boundary.CanonicalIds`
3. `PovuLean.Algorithms.Flubble.Boundary.ordered`
4. `PovuLean.Algorithms.Flubble.CanonicalClassBoundary`
5. `PovuLean.Algorithms.Flubble.IsPaperBoundary`
6. `PovuLean.Algorithms.Flubble.IsFlubbleBoundary`
7. `PovuLean.Algorithms.Flubble.NoDuplicateBoundaries`

Module: `PovuLean.Algorithms.Flubble.InputInvariant`

Key names:

1. `PovuLean.Algorithms.Flubble.IsBoundaryCandidate`
2. `PovuLean.Algorithms.Flubble.candidateStack`
3. `PovuLean.Algorithms.Flubble.SupportedInput`
4. `PovuLean.Algorithms.Flubble.CycleClassAssignment`
5. `PovuLean.Algorithms.Flubble.CycleClassAssignment.Correct`
6. `PovuLean.Algorithms.Flubble.SupportedGFAInput`
7. `PovuLean.Algorithms.Flubble.SupportedGFAInput.toSupportedInput`

These definitions establish the semantic input model: a supported frame, a candidate stack of eligible real black tree links, and a correct cycle-class assignment.

3.2. Detector Correctness And Bounds

Module: `PovuLean.Algorithms.Flubble.Detect`

Key names:

1. `PovuLean.Algorithms.Flubble.detectFlubbles`
2. `PovuLean.Algorithms.Flubble.detectFlubbles_length_le_candidateStack_length`
3. `PovuLean.Algorithms.Flubble.detectFlubbles_length_le_treeLinks_length`
4. `PovuLean.Algorithms.Flubble.detectFlubbles_length_le_graph_links_length`
5. `PovuLean.Algorithms.Flubble.detectFlubbles_length_le_graph_edgeCount`
6. `PovuLean.Algorithms.Flubble.detectFlubbles_length_le_supportedInput_graph_edgeCount`
7. `PovuLean.Algorithms.Flubble.detectFlubbles_noDuplicates`

Module: `PovuLean.Algorithms.Flubble.Correctness`

Key names:

1. `PovuLean.Algorithms.Flubble.detectFlubbles_sound`
2. `PovuLean.Algorithms.Flubble.detectFlubbles_complete`
3. `PovuLean.Algorithms.Flubble.detectFlubbles_canonical_noDuplicates`
4. `PovuLean.Algorithms.Flubble.detectFlubbles_correct`
5. `PovuLean.Algorithms.Flubble.detectFlubbles_correct_for_gfa`

These theorems prove that the Lean reference detector is sound and complete for the flubble boundary relation, emits canonical duplicate-free boundaries, and has output size bounded by the candidate stack and graph edge count.

3.3. Indexed Detector And Equivalence

Module: `PovuLean.Algorithms.Flubble.Indexed`

Key semantic-equivalence names:

1. `PovuLean.Algorithms.Flubble.lookup_indexedEntriesFrom_eq_firstSameClass?`
2. `PovuLean.Algorithms.Flubble.indexedBoundaryAt?_eq_closeAfterGap?`
3. `PovuLean.Algorithms.Flubble.detectStackIndexedRaw_eq_detectStackRaw`
4. `PovuLean.Algorithms.Flubble.detectStackIndexed_eq_detectStack`
5. `PovuLean.Algorithms.Flubble.detectFlubblesIndexed_eq_detectFlubbles`
6. `PovuLean.Algorithms.Flubble.detectFlubblesIndexed_iff_isFlubbleBoundary`
7. `PovuLean.Algorithms.Flubble.detectFlubblesIndexed_noDuplicates`

Key operation-count names:

1. `PovuLean.Algorithms.Flubble.scanStackIndexedAux_visited`
2. `PovuLean.Algorithms.Flubble.scanStackIndexedAux_lookups`
3. `PovuLean.Algorithms.Flubble.scanStackIndexedAux_updates`
4. `PovuLean.Algorithms.Flubble.detectStackIndexedRaw_one_lookup_and_update_per_candidate`

These theorems justify replacing the reference scan by an indexed scan without changing the emitted boundaries, while exposing the per-candidate operation counts needed by the linear-time cost theorem.

3.4. Hierarchy Correctness

Module: `PovuLean.Algorithms.FlubbleTree.Spec`

Key names:

1. `PovuLean.Algorithms.FlubbleTree.Span`
2. `PovuLean.Algorithms.FlubbleTree.boundarySpan?`
3. `PovuLean.Algorithms.FlubbleTree.SupportedHierarchyInput`
4. `PovuLean.Algorithms.FlubbleTree.UnsupportedHierarchyInput`
5. `PovuLean.Algorithms.FlubbleTree.IsForestHierarchy`
6. `PovuLean.Algorithms.FlubbleTree.IsCorrectHierarchy`

Module: `PovuLean.Algorithms.FlubbleTree.Correctness`

Key names:

1. `PovuLean.Algorithms.FlubbleTree.buildHierarchyFrom_correct`
2. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_correct`
3. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_rejects_unsupported`
4. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_nodes_length`
5. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_nodes_length_le_detectFlubbles_length`
6. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_nodes_length_le_supportedInput_graph_edgeCount`
7. `PovuLean.Algorithms.FlubbleTree.buildHierarchy_flatMetadataSlots_le_supportedInput_graph_edgeCount_twice`

These theorems prove that supported laminar boundary spans produce the canonical flat hierarchy and that hierarchy output metadata is linear in the detector output and, under supported input, graph edge count.

3.5. Cost And Linearity Theorems

Module: `PovuLean.Complexity.Flubble`

Key names:

1. `PovuLean.Complexity.Flubble.GraphSizes`
2. `PovuLean.Complexity.Flubble.DecompositionSizes`
3. `PovuLean.Complexity.Flubble.StageCost`
4. `PovuLean.Complexity.Flubble.ClassIndexContract`
5. `PovuLean.Complexity.Flubble.IndexedDetectorOperationContract`
6. `PovuLean.Complexity.Flubble.IndexedDetectorOperationContract.operationCost_linear_in_candidates`
7. `PovuLean.Complexity.Flubble.ConditionalExtractionCostContract`
8. `PovuLean.Complexity.Flubble.boundaryOutputSize_linear_in_candidateStack`
9. `PovuLean.Complexity.Flubble.hierarchyConstructionOutputSize_linear_in_candidateStack`
10. `PovuLean.Complexity.Flubble.conditional_extraction_linear_in_candidateStack`
11. `PovuLean.Complexity.Flubble.DecompositionStageCosts.total_linear_with`

Module: `PovuLean.Complexity.Decomposition`

Key names:

1. `PovuLean.Complexity.Decomposition.SemanticDecomposeIntermediateSizes`
2. `PovuLean.Complexity.Decomposition.SemanticDecomposeIntermediateSizeBounds`
3. `PovuLean.Complexity.Decomposition.SemanticDecomposePipelineCosts`
4. `PovuLean.Complexity.Decomposition.SemanticDecomposeStageContracts`
5. `PovuLean.Complexity.Decomposition.conditional_semantic_decompose_linear`

The `conditional_semantic_decompose_linear` theorem composes stage linearity, intermediate-size bounds, flubble extraction linearity, and semantic VCF correctness into one proof-side statement.

3.6. Semantic Pipeline Result

Module: `PovuLean.Pipeline`

Key name:

1. `PovuLean.Pipeline.semanticGfaToVcf_correct`

This theorem is the semantic GFA-to-VCF join point. Given accepted GFA input, supported flubble input, correct cycle classes, correct hairpin scan, supported hierarchy input, and semantic variant-call witnesses, emitted VCF records are correct.

4. Conditional Linear-Time Theorem

The strongest current full-pipeline theorem is:

```
PovuLean.Complexity.Decomposition.conditional_semantic_decompose_linear
```

Its conclusion is:

```
LinearBound costs.total sizes.inputGraphSize
  /\ VCF.EmissionCorrect frame classes scan
    (Algorithms.FlubbleTree.buildHierarchy frame classes)
    calls
    (VCF.emitRecords calls)
```

The theorem receives these stage contracts through `SemanticDecomposeStageContracts`:

1. `componentDecompositionLinear`,
2. `tipDummyAugmentationLinear`,
3. `traversalFrameConstructionLinear`,
4. `cycleClassAssignmentLinear`,
5. `hairpinScanLinear`, and
6. `flubbleExtractionAndHierarchy`.

It also receives `SemanticDecomposeIntermediateSizeBounds`, which is the proof that component graph size, augmented graph size, traversal-frame size, candidate-stack size, and hairpin-boundary count are each linear in the input graph-size measure.

The flubble portion is delegated to:

```
PovuLean.Complexity.Flubble.conditional_extraction_linear_in_candidateStack
```

That theorem assumes:

1. constant class-index lookup/update costs,
2. exact lookup/update counts from the indexed detector theorem,
3. scan bookkeeping linear in the candidate stack,
4. boundary emission linear in emitted boundaries,
5. hierarchy construction linear in hierarchy output size, and
6. supported laminar hierarchy input.

Therefore the correct claim is: Lean proves conditional semantic linearity under named contracts. The incorrect claim would be: current arbitrary C++ povu executions have already been formally proved to run in linear time.

5. C++ Bridge And Conformance Evidence

The implementation bridge is evidence, not a formal refinement theorem. Relevant documentation and surfaces are:

1. docs/lean4-proof/bridge_cost_instrumentation.md
2. docs/lean4-proof/bridge_povu_implementation.md
3. docs/lean4-proof/checked_translator.md
4. src/povu/common/stage_cost.cpp
5. include/povu/common/stage_cost.hpp
6. tests/lean4_conformance/src/main.rs
7. src/mto/to_structure_export.cpp
8. povu-rs/src/native_gfa.rs
9. povu-rs/tests/native_gfa_vcf_tests.rs

5.1. Cost Instrumentation

The opt-in environment variable `POVU_STAGE_COST_TRACE=1` emits stage-counter lines with stable contract= identifiers. These identifiers align implementation trace points with the Lean stage slots:

1. componentDecomposition
2. tipDummyAugmentation
3. traversalFrameConstruction
4. cycleClassAssignment
5. boundaryEmission
6. hierarchyConstruction

The counters report calls, size proxies, outputs, and elapsed nanoseconds. They are useful bridge artifacts, but elapsed time and container behavior are not Lean cost witnesses.

5.2. Flubble Export And Structure Evidence

The C++ conformance path exposes canonical structure information through `--structure-export` in the CLI and `src/mto/to_structure_export.cpp`. The bridge documentation describes exported stack/class/hierarchy information used by fixture-level checks. The Rust semantic port in `povu-rs/src/native_gfa.rs` contains `detect_flubble_stack`, with tests for close-after-gap, first-later-same-class, canonicalization, and duplicate removal.

This is conformance evidence over maintained fixtures and port-level tests. It does not quantify over all supported graphs.

5.3. Checked Translator

The checked-translator artifact is produced by the Lean conformance harness:

```
cargo run --manifest-path tests/lean4_conformance/Cargo.toml -- \
  --repo-root . --fixture minimal-substitution \
  --checked-translator build/lean4-conformance/minimal-substitution-witness.json
```

Its schema is `povu.lean4.checked-translator.v1`. It records the fixture, current povu command, Lean theorem boundary, trusted assumptions, exported C++ structure, Lean reference structure, and normalized semantic VCF witness. The artifact targets `PovuLean.Pipeline.semanticGfaToVcf_correct`; it does not synthesize a Lean proof term.

5.4. Cycle Class Checker

The bridge audit records a direct fixture checker for C++ class ids against an independent cycle-equivalence oracle. It tests both directions of the Lean `CycleClassAssignment`. Correct obligation on selected small supported structure fixtures:

1. same exported C++ class id implies same oracle group, and
2. same oracle group implies same exported C++ class id.

This is strong finite conformance evidence. It is not a theorem for arbitrary supported inputs.

5.5. Hierarchy Span Checker

The bridge audit also records a PVST parent-edge checker. It reconstructs flubble boundaries from exported stack entries using the Lean first-later same-class nonempty-gap rule, computes stack spans, derives nearest strict span-containment parents, and compares them with exported PVST parents.

Again, the result is fixture-level conformance evidence for exported stacks, not a general proof of C++ hierarchy construction.

6. Remaining Open Gap

A full arbitrary C++ runtime refinement proof still needs a machine-checkable bridge from implementation executions to the Lean witnesses and cost contracts. The missing items are:

1. parser refinement from accepted byte-level GFA records to Lean `GFA.Document.Accepted` and `GFA.Document.toGraph`;
2. component-decomposition refinement showing C++ components preserve Lean graph semantics and have total size linear in the input;
3. traversal-frame refinement exporting the C++ spanning tree, tip/dummy augmentation, ordered tree links, real/black flags, and candidate-stack order as a Lean `TraversalFrame`;
4. class refinement proving C++ bracket classes satisfy `CycleClassAssignment.Correct` for arbitrary supported inputs;
5. extraction refinement proving C++ `next_seen` and flubble emission match Lean `closeAfterGap?`, `firstSameClass?`, `Boundary.ordered`, and `detectFlubbles`;
6. hierarchy refinement proving C++ PVST parent edges match Lean nearest strict stack-span containment for arbitrary supported laminar inputs;
7. a concrete runtime cost model for loops, allocations, serialization, FFI, and container operations such as `std::vector`, `std::map`, `std::unordered_map`, `std::unordered_set`, `std::set`, `std::list`, Rust collections, and serialization buffers; and
8. proof that observed or exported stage counters instantiate the Lean `StageCost`, `ExtractionPipelineCosts`, and `SemanticDecomposePipelineCosts` records.

Until those obligations are discharged, the precise project status is:

“Lean proves semantic detector correctness, hierarchy correctness, indexed detector equivalence, output-size bounds, and conditional linear-time composition. Current povu has bridge artifacts and conformance evidence for selected implementation surfaces. A formal arbitrary C++ runtime refinement proof remains open.”

7. Build And Reference Check

The rendered PDF for this source is:

```
docs/lean4-proof/flubble_linear_proof.pdf
```

Build command:

```
typst compile docs/lean4-proof/flubble_linear_proof.typ \
docs/lean4-proof/flubble_linear_proof.pdf
```

The Lean names listed above are intended to be checked with a temporary `#check` file importing:

```
import PovuLean.Algorithms.Flubble.Correctness
import PovuLean.Algorithms.Flubble.Indexed
import PovuLean.Algorithms.FlubbleTree.Correctness
import PovuLean.Complexity.Flubble
import PovuLean.Complexity.Decomposition
import PovuLean.Pipeline
```